# Atmosphere Direction Not Syncing to Clients - Bug Report

## Summary

Wind direction, atmospheric particles, and wind sounds do not work correctly for clients in multiplayer games (both player-hosted and dedicated servers). The effects work initially when joining, but become stale/incorrect when wind direction changes during gameplay.

## Symptoms

- Wind particles move in wrong/outdated directions for clients
- Wind sounds don't play when they should
- Atmospheric effects initially work when joining server, then stop updating
- Sealed rooms continue showing atmospheric animations indefinitely
- Host/server sees correct behavior, clients do not

## Root Cause Analysis

### What's Happening

In `AtmosphereHelper.ReadStatic()`, when the server sends atmosphere updates to clients:

1. ✅ The `Direction` vector **IS** read from the network stream (when flag 8 is set)
2. ✅ For **NEW** atmospheres, Direction **IS** applied correctly
3. ❌ For **EXISTING** atmospheres, Direction is **READ but DISCARDED** - never applied!

### The Bug Location

**File:** `Assets/Scripts/Atmospherics/AtmosphereHelper.cs`
**Method:** `ReadStatic(RocketBinaryReader reader)`
**Line:** ~665-681

# Current (Broken) Code

```
public static void ReadStatic(RocketBinaryReader reader)
{
    // ... read header ...
    long referenceId;
    Network.ReadPackedId(reader, out referenceId);
    byte networkUpdateFlags = reader.ReadByte();
    AtmosphereMode mode = (AtmosphereMode)reader.ReadByte();

    // Try to find existing atmosphere
    Atmosphere atmosphere = null;
    if (referenceId > 0L)
    {
        atmosphere = Referencable.Find<Atmosphere>(referenceId);
    }

    // ... read parent reference ...

    // Read Direction from network stream
    WorldGrid worldGrid = WorldGrid.INVALID;
    Vector3 direction = Vector3.zero;

    if (mode == AtmosphereMode.World || mode == AtmosphereMode.Thing)
    {
        if (IsNetworkUpdateRequired(2, networkUpdateFlags))
        {
            worldGrid = reader.ReadWorldGrid();
        }
        if (IsNetworkUpdateRequired(8, networkUpdateFlags))  // ← Direction IS read here
        {
            direction = reader.ReadVector3Half();
        }
    }

    // Create new atmosphere if needed
    if (atmosphere == null && referenceId > 0L)
    {
        switch (mode)
        {
            case AtmosphereMode.World:
                atmosphere = new Atmosphere(worldGrid, referenceId)
                {
```

```
                    Direction = direction  // ← NEW atmospheres GET the Direction
                };
                break;
            // ... other cases ...
        }
    }

    // ... set basic properties ...

    // ❌ BUG: Direction is NEVER applied to EXISTING atmospheres!
    // The 'direction' variable is read from the network but then thrown away

    atmosphere.Read(reader, networkUpdateFlags);
}
```

## Why This Causes the Symptoms

1. **Initial Join (Works):** Client loads area → atmospheres created as NEW → Direction applied → effects work ✅
2. **Direction Changes (Breaks):** Wind shifts, doors open, vents activate → server sends Direction update (flag 8) → client reads it but doesn't apply it → Direction stays stale → effects wrong ❌
3. **Sealed Room Animation Bug:** Room sealed → server sends `Direction ≈ Vector3.zero` → client ignores it → stale Direction remains → particles never fade out ❌

# The Fix

## Required Code Change

**File:** `Assets/Scripts/Atmospherics/AtmosphereHelper.cs`
**Method:** `ReadStatic(RocketBinaryReader reader)`

Add these lines **AFTER** setting basic properties but **BEFORE** calling `atmosphere.Read()` :

```
// Set basic properties
atmosphere.ReferenceId = referenceId;
atmosphere.Mode = mode;

// *** ADD THIS FIX ***
// Apply Direction to EXISTING atmospheres too (not just new ones)
if (IsNetworkUpdateRequired(8, networkUpdateFlags) &&
    (mode == AtmosphereMode.World || mode == AtmosphereMode.Thing))
{
    atmosphere.Direction = direction;
}

// Read remaining data
atmosphere.Read(reader, networkUpdateFlags);
```

# Complete Fixed Method

```
public static void ReadStatic(RocketBinaryReader reader)
{
    // Read the header
    long referenceId;
    Network.ReadPackedId(reader, out referenceId);
    byte networkUpdateFlags = reader.ReadByte();
    AtmosphereMode mode = (AtmosphereMode)reader.ReadByte();

    // Try to find existing atmosphere
    Atmosphere atmosphere = null;
    if (referenceId > 0L)
    {
        atmosphere = Referencable.Find<Atmosphere>(referenceId);
    }

    // Read parent reference if flag 1 is set
    long parentReferenceId = 0L;
    if (IsNetworkUpdateRequired(1, networkUpdateFlags))
    {
        Network.ReadPackedId(reader, out parentReferenceId);
    }

    // Read WorldGrid and Direction for World/Thing modes
    WorldGrid worldGrid = WorldGrid.INVALID;
    Vector3 direction = Vector3.zero;

    if (mode == AtmosphereMode.World || mode == AtmosphereMode.Thing)
    {
        if (IsNetworkUpdateRequired(2, networkUpdateFlags))
        {
            worldGrid = reader.ReadWorldGrid();
        }
        if (IsNetworkUpdateRequired(8, networkUpdateFlags))
        {
            direction = reader.ReadVector3Half();
        }
    }

    // Create new atmosphere if needed
    if (atmosphere == null && referenceId > 0L)
    {
```

```
switch (mode)
{
    case AtmosphereMode.World:
    {
        Atmosphere existingAtmo = AtmosphericsManager.Find(worldGrid);
        if (existingAtmo != null)
        {
            AtmosphericsManager.AllAtmospheres.Remove(existingAtmo);
        }
        atmosphere = new Atmosphere(worldGrid, referenceId)
        {
            Direction = direction
        };
        break;
    }
    case AtmosphereMode.Network:
    {
        AtmosphericsNetwork network = Referencable.Find<AtmosphericsNetwork>(parentRefe
        if (network == null)
        {
            atmosphere = new Atmosphere();
        }
        else
        {
            atmosphere = new Atmosphere(network, referenceId);
            network.AssignAtmosphere(atmosphere);
        }
        break;
    }
    case AtmosphereMode.Thing:
    {
        Thing thing = Referencable.Find<Thing>(parentReferenceId);
        if (thing == null)
        {
            atmosphere = new Atmosphere();
        }
        else
        {
            if (thing.InternalAtmosphere != null)
            {
                thing.InternalAtmosphere.Thing = null;
            }
            atmosphere = new Atmosphere(thing, new VolumeLitres(1.0), referenceId);
```

```
                    thing.InternalAtmosphere = atmosphere;
                }
                break;
            }
            default:
                atmosphere = new Atmosphere();
                break;
        }
    }

    // Fallback if still null
    if (atmosphere == null)
    {
        atmosphere = new Atmosphere();
    }

    // Set basic properties
    atmosphere.ReferenceId = referenceId;
    atmosphere.Mode = mode;

    // *** FIX: Apply Direction to EXISTING atmospheres ***
    if (IsNetworkUpdateRequired(8, networkUpdateFlags) &&
        (mode == AtmosphereMode.World || mode == AtmosphereMode.Thing))
    {
        atmosphere.Direction = direction;
    }

    // Read remaining data
    atmosphere.Read(reader, networkUpdateFlags);
}
```

# Verification

## Server Already Works Correctly

The server-side serialization in `Atmosphere.Write()` is **already correct**:

```
public void Write(RocketBinaryWriter writer, byte networkUpdateFlags)
{
    // ... other data ...

    if (IsNetworkUpdateRequired(8, networkUpdateFlags))
    {
        writer.WriteVector3Half(this.Direction);  // ✅ Server sends Direction
    }

    // ... remaining data ...
}
```

The issue is purely in client-side **deserialization**.

## How to Test the Fix

1. **Multiplayer Test:**
   - Host or join a dedicated server
   - Observe wind particles and atmospheric effects
   - Open/close doors, activate vents to change air flow
   - Verify wind direction updates correctly for clients
2. **Sealed Room Test:**
   - Build a room with atmospheric effects visible
   - Open it to space/vent it out to create air flow
   - Seal the room completely
   - Verify atmospheric particles fade out within 10-30 seconds
3. **Dedicated Server Test:**
   - Connect as client to dedicated server
   - Verify all atmospheric effects work consistently
   - Move between areas and verify effects update properly

# Impact

- **Priority:** Medium-High (affects immersion and gameplay feedback)
- **Affected Systems:** Wind particles, atmospheric sounds, fog effects, fire visuals
- **Multiplayer Only:** Single-player is unaffected (local simulation works correctly)
- **Performance Impact:** None (fix only applies existing data that's already being transmitted)

# Additional Notes

## Why This Bug Went Unnoticed

The bug is subtle because:

1. Effects **do** work initially when joining (new atmospheres are created)
2. Only **updates** are broken (atmosphere already exists)
3. It appears intermittent depending on when/where you join
4. Single-player works fine (no network serialization)

## Network Traffic

The fix requires **zero additional network bandwidth** - the Direction data is already being transmitted by the server. The bug is purely that the client throws away the data instead of using it.

## Related Code

The Direction vector is used by:

- `AtmosphericsManager.ParticleThread()` - calculates particle velocities
- `AtmosphericAudioHandler` - determines wind sound playback
- Particle fade logic - determines when to stop/fade effects

All of these systems currently work correctly **if** Direction is properly synchronized.

# BepInEx Workaround Mod

A temporary fix is available as a BepInEx plugin that patches the issue using Harmony. However, the proper fix should be implemented in the base game code as shown above.

**Mod Repository:** Available on request
**Installation:** BepInEx/plugins/ folder (client-side only)